Συστήματα Διαχείρισης και Ανάλυσης Δεδομένων
Διδάσκων: Ιωάννης Κωτίδης

Εαρινό εξάμηνο 2022-2023

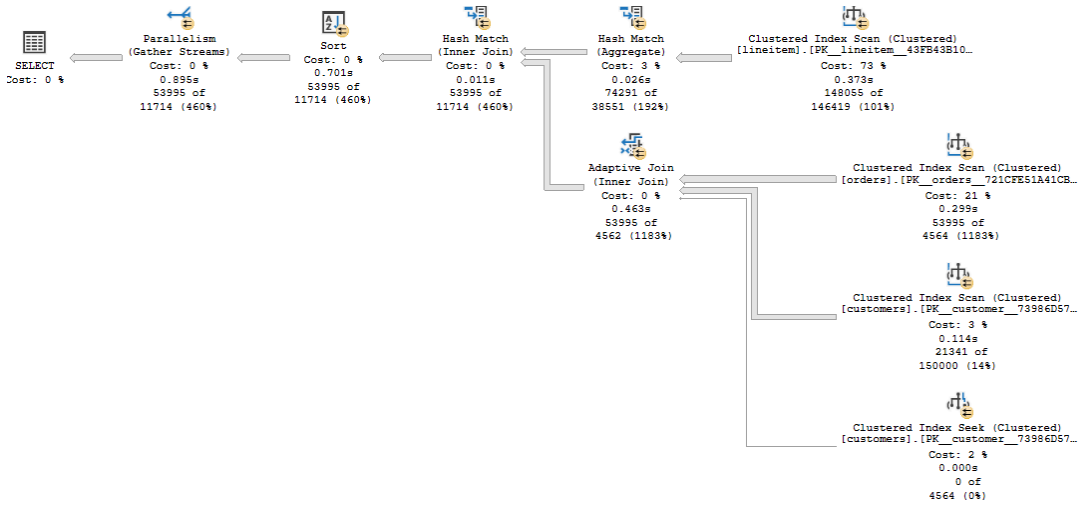
Ενδεικτικές Λύσεις Πρώτης Εργασίας

Λύση Πρώτου Ζητήματος

Στατιστικά IO και πλάνο εκτέλεσης του επερωτήματος πριν τη δημιουργία ευρετηρίων.

```
(53995 rows affected)
Table 'customers'. Scan count 5, logical reads 2685, physical reads 2, page
server reads 0, read-ahead reads 2562, ....
Table 'orders'. Scan count 5, logical reads 16961, physical reads 1, page
server reads 0, read-ahead reads 16729, …..
Table 'lineitem'. Scan count 5, logical reads 60383, physical reads 1, page
server reads 0, read-ahead reads 59296 ……


(1 row affected)
```



Δημιουργία ευρετηρίων

```
create index Q1_idx1 ON orders(orderdate) include (custkey)
create index Q1_idx2 ON lineitem(shipdate) include (price)
```

Δημιουργούμε το ευρετήριο **Q1_idx1** στο πεδίο **orders.orderdate** διότι αποτελεί κριτήριο αναζήτησης. Στο ευρετήριο περιλαμβάνουμε και το πεδίο custKey ώστε να επιταχύνουμε την σύζευξη των πινάκων customers και orders (**customers.custkey=orders.custkey**).

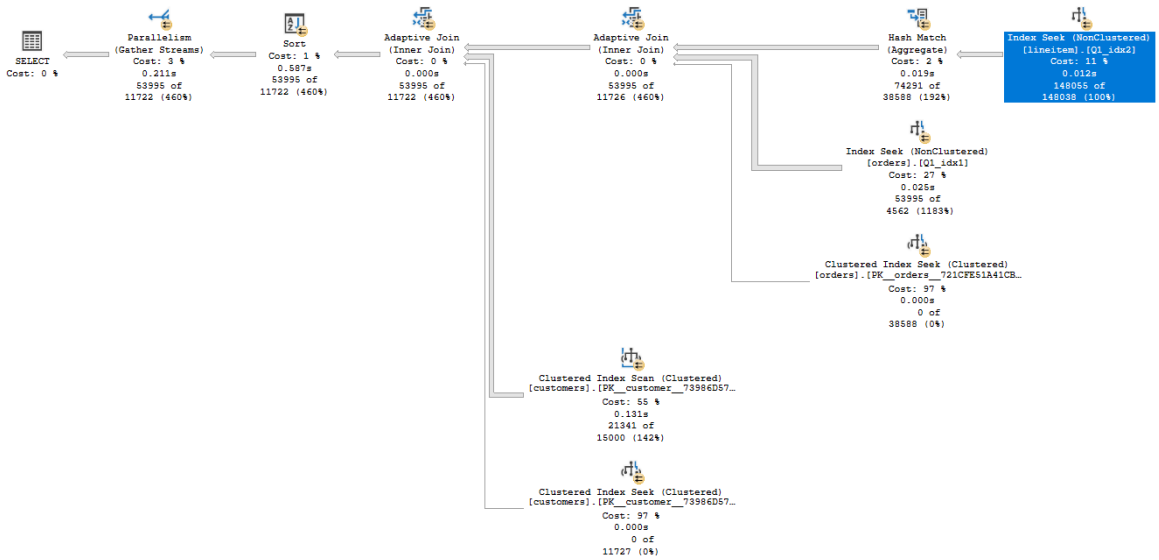
Δημιουργούμε το ευρετήριο **Q1_idx2** στο πεδίο **lineitem.shipdate** διότι αποτελεί κριτήριο αναζήτησης. Στο ευρετήριο περιλαμβάνουμε και το πεδίο **price** ώστε να επιταχύνουμε την δημιουργία του αθροίσαματος **SUM(price).**

Στατιστικά IO και πλάνο εκτέλεσης του επερωτήματος μετά από τη δημιουργία ευρετηρίων.

```
53995 rows affected)
Table 'customers'. Scan count 5, logical reads 2685, physical reads 2, page
server reads 0, read-ahead reads 2562, page server read-ahead reads 0 ….
Table 'orders'. Scan count 5, logical reads 1008, physical reads 1, page server
reads 0, read-ahead reads 959, page server read-ahead reads 0, ….
Table 'lineitem'. Scan count 5, logical reads 428, physical reads 1, page
server reads 0, read-ahead reads 403, ….

(1 row affected)
```

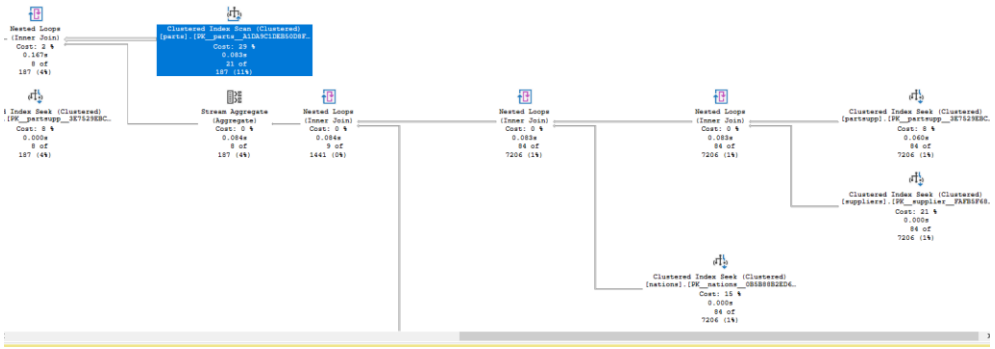
Παρατηρούμε σημαντική μείωση στον αριθμό των σελίδων που διαβάζονται από τους πίνακες orders και lineitem με την δημιουργία των ευρετηρίων. Στο πλάνο ακόλουθο πλάνο εκτέλεσης βλέπουμε ότι και τα δύο ευρετήρια αξιοποιούνται στην εκτέλεση του επερωτήματος με βέλτιστο τρόπο (index seek).



Λύση Δεύτερου Ζητήματος

Στατιστικά IO και πλάνο εκτέλεσης του επερωτήματος πριν τη δημιουργία ευρετηρίων.

```
Table 'suppliers'. Scan count 1, logical reads 408, physical reads 1, page
server reads 0, read-ahead reads 168,
Table 'nations'. Scan count 1, logical reads 170, physical reads 1, page server
reads 0, read-ahead reads 0,
Table 'regions'. Scan count 1, logical reads 170, physical reads 1, page server
reads 0, read-ahead reads 0,
Table 'partsupp'. Scan count 29, logical reads 108, physical reads 29, page
server reads 0, read-ahead reads 0,
Table 'parts'. Scan count 1, logical reads 2795, physical reads 2, page server
reads 0, read-ahead reads 2798,
```

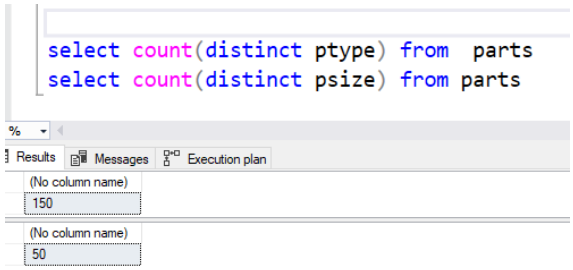


Δημιουργία ευρετηρίων

Από τα παραπάνω στοιχεία παρατηρούμε ότι το μεγαλύτερο κόστος στην εκτέλεση του ερωτήματος προέρχεται από την ανάγνωση των σελίδων του πίνακα parts. Για το λόγο αυτό δημιουργούμε το παρακάτω ευρετήριο:

```
create index Q2_idx1 ON parts(ptype,psize) include (manufacturer)
```

Θέτουμε πρώτο πεδίο στο ευρετήριο το πεδίο **ptype** αντί του **psize** για να έχει το ευρετήριο μεγαλύτερη επιλεξιμότητα (υπάρχουν 150 διαφορετικές τιμές για το πεδίο ptype και μόνο 50 για το psize).

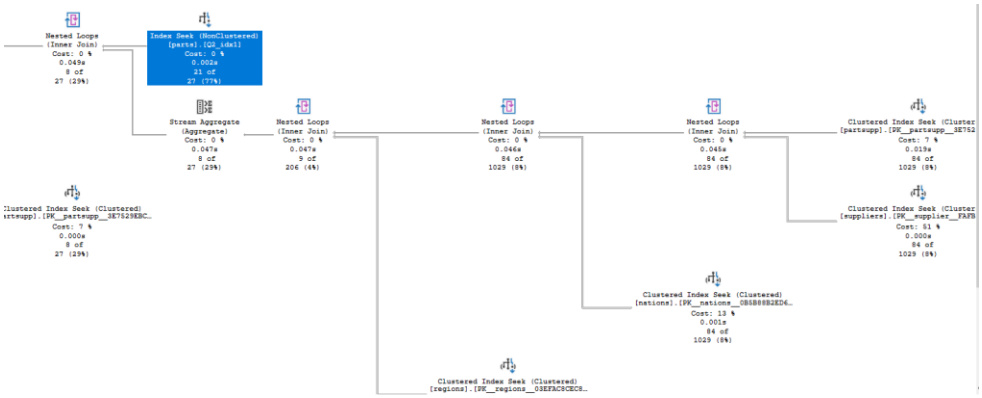


Οι πίνακες nations και regions περιέχουν 25 και 5 εγγραφές αντίστοιχα οπότε η δημιουργία ευρετηρίων δεν έχει νόημα.

Στατιστικά IO και πλάνο εκτέλεσης του επερωτήματος μετά από τη δημιουργία ευρετηρίων.

```
Table 'regions'. Scan count 0, logical reads 184, physical reads 1, page server
reads 0, read-ahead reads 0
Table 'nations'. Scan count 0, logical reads 184, physical reads 1, page server
reads 0, read-ahead reads 0
Table 'suppliers'. Scan count 0, logical reads 272, physical reads 1, page
server reads 0, read-ahead reads 168
Table 'partsupp'. Scan count 29, logical reads 108, physical reads 29, page
server reads 0, read-ahead reads 0
Table 'parts'. Scan count 1, logical reads 3, physical reads 3, page server
reads 0, read-ahead reads 0
```

Παρατηρούμε σημαντική μείωση στον αριθμό των σελίδων που διαβάζονται από τον πίνακα parts. Στο ακόλουθο πλάνο εκτέλεσης βλέπουμε ότι το ευρετήριο που δημιουργήσαμε χρησιμοποιείται για την εκτέλεση του επερωτήματος (index seek).

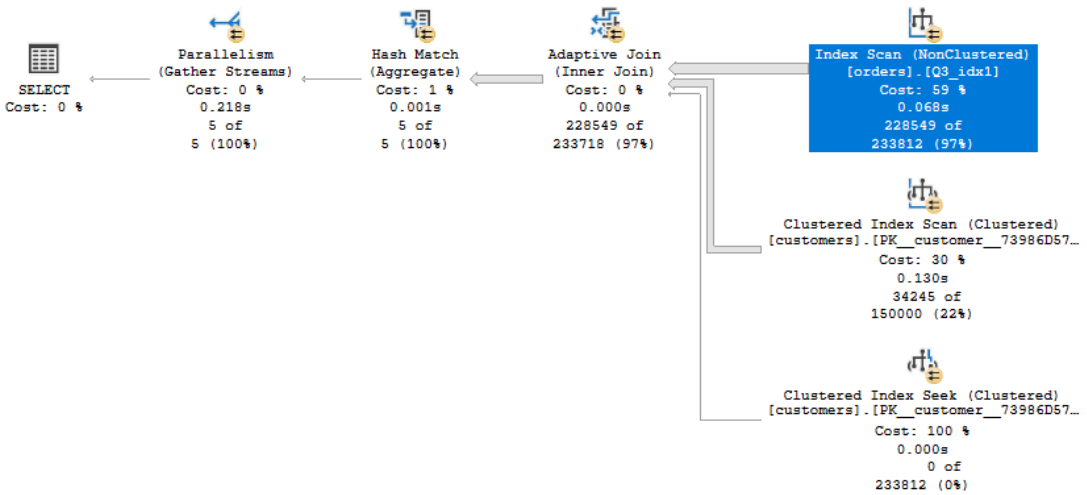


Λύση Τρίτου Ζητήματος

Το ερώτημα είναι σχετικά απλό και η πρώτη μας σκέψη είναι να δημιουργήσουμε ένα ευρετήριο στο κριτήριο αναζήτησης orderdate. Για να επιταχύνουμε την δημιουργία του αθροίσματος περιλαμβάνουμε στο ευρετήριο και το πεδίο totalprice και αντίστοιχα το πεδίο custkey για να επιταχύνουμε την ισοσύνδεση του πίνακα orders με τον πίνακα customers.

```
create index Q3_idx1 ON orders(orderdate) include (custkey, totalprice)
```

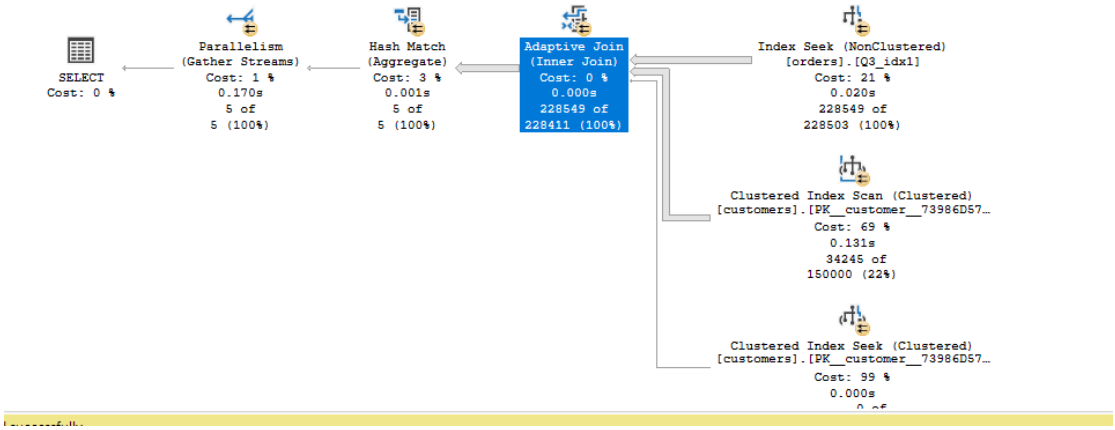
Μετά την δημιουργία του ευρετηρίου εκτελούμε το επερώτημα και εξετάζουμε το πλάνο εκτέλεσης.



Παρατηρούμε ότι το ευρετήριο που δημιουργήσαμε δεν αξιοποιείται με τον καλύτερο τρόπο (INDEX SCAN αντί για INDEX SEEK) για την εκτέλεση του επερωτήματος. Ο λόγος είναι η εφαρμογή της συνάρτησης YEAR στο κριτήριο αναζήτησης.

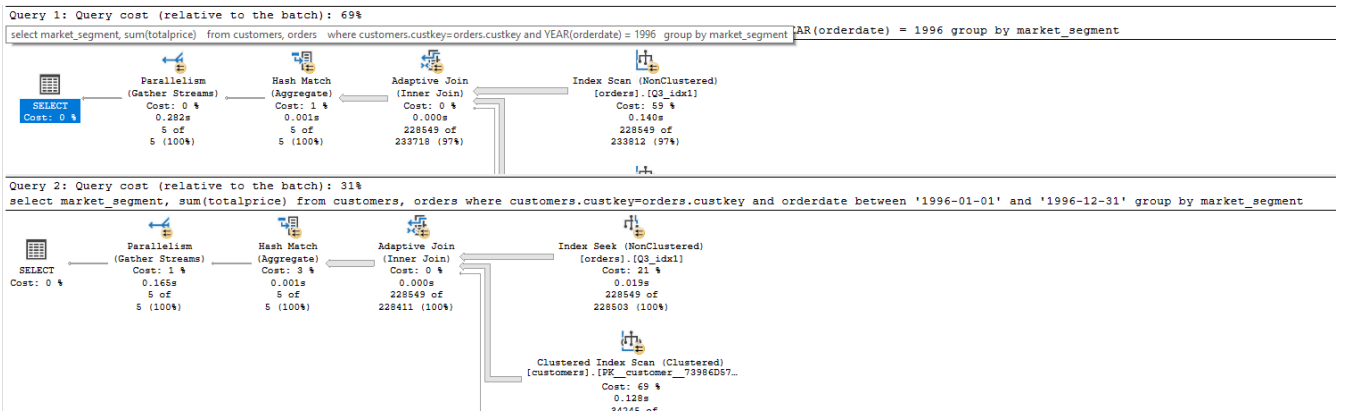
Μπορούμε να πάρουμε το ίδιο αποτέλεσμα εκτελώντας το παρακάτω επερώτημα:

```
select market_segment, sum(totalprice)
   from customers, orders
  where customers.custkey=orders.custkey and YEAR(orderdate) = 1996
  group by market_segment
```



Παρατηρούμε ότι το ευρετήριο αξιοποιείται με βέλτσιστο τρόπο (INDEX SEEK)

Εκτελούμε τα δύο επερωτήματα σε δέσμη και βλέπουμε τη διαφορά.



Λύση

1. Εξετάζοντας τα πλάνα εκτέλεσης και των ερωτημάτων πριν και μετά την δημιουργία των ευρετηρίων Q4_idx1 και Q4_idx2 βλέπουμε ότι τα ευετήρια βλετιώνουν την απόδοση και των δύο επερωτημάτων

2.

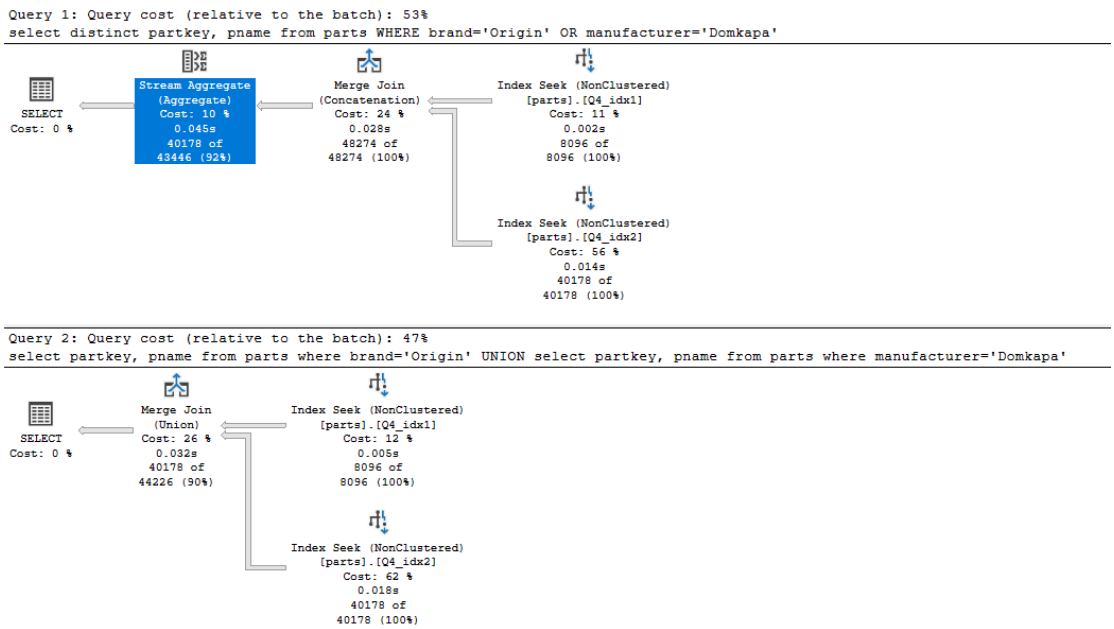
Α) Γράφουμε την παρακάτω εναλλακτική μορφή του επερωτήματος **a.**

```
select partkey, pname
 from parts
where brand='Origin'

UNION

select partkey, pname
 from parts
where manufacturer='Domkapa'
```

Εκτελούμε τα δύο επερωτήματα σε δέσμη και συγκρίνουμε τα πλάνα εκτέλεσης. Βλέπουμε ότι το υποερώτημα με τον τελεστή UNION εκτελείται ταχύτερα. Το κόστος IO είναι το ίδιο και για τα δύο επερωτήματα. Η διαφορά έγκειτε στο κόστος υπολογισμού του τελεστή DISTINCT του πρώτου επερωτήματος.



B) Γράφουμε την παρακάτω εναλλακτική μορφή του επερωτήματος **b.**

```
select partkey, pname
 from parts
where brand='Origin' and manufacturer='Domkapa'
```

και δημιουργούμε το ακόλουθο ευρετήριο

```
create index Q4_idx3 on parts (brand, manufacturer) include (pname)
```

Εκτελούμε τα δύο επερωτήματα σε δέσμη και συγκρίνουμε τα πλάνα εκτέλεσης. Βλέπουμε ότι το κόστος εκτέλεσης του νέου επερωτήμαος σε συνδυασμό με το ευρετήριο Q4_idx3 είναι σημαντικά μικρότερο σε σχέση με το αρχικό επερώτημα.

